Method:
For this lab I used dynamic, bottom up approach for the probability calculation and then used that within my payout function.

Approach:
Initially I just wrote the naive solution in order to understand the basic concept of the problem. Then I began work on the dp solution which involves keeping track of a 2D table where each index stores the probability of P(i, j, n). The base cases are when A has hit the target or B has hit the target. Then it computes each entry for the table from the top (n, n) down to (0, 0) and then returns the table entry at the desired location. The payout function just uses that probability function to split the pot based on the likelihood of each person winning.

Reasoning:
This algorithm is efficient because the dp table allows for each entry to be calculated before it is needed. It also avoids repeated calculation by storing the earlier computed entries in the table.

Reflection:
I used these test cases:

prob_rec:
Example 1: prob_rec(8, 6, 10) = 0.8125 (Player A wins 81.25% of the time)
Example 2: prob_rec(9, 8, 10) = 0.75
Example 3: prob_rec(0, 0, 3) = 0.5

prob_dp:
prob_dp(8, 6, 10) = 0.8125
prob_dp(9, 8, 10) = 0.75
prob_dp(0, 0, 1) = 0.5
prob_dp(0, 0, 3) = 0.5

and payout:
payout(8, 6, 10, 100) = (81.25, 18.75)
payout(9, 8, 10, 200) = (150.0, 50.0)

Complexity:
O(n^2) (Time and Space)